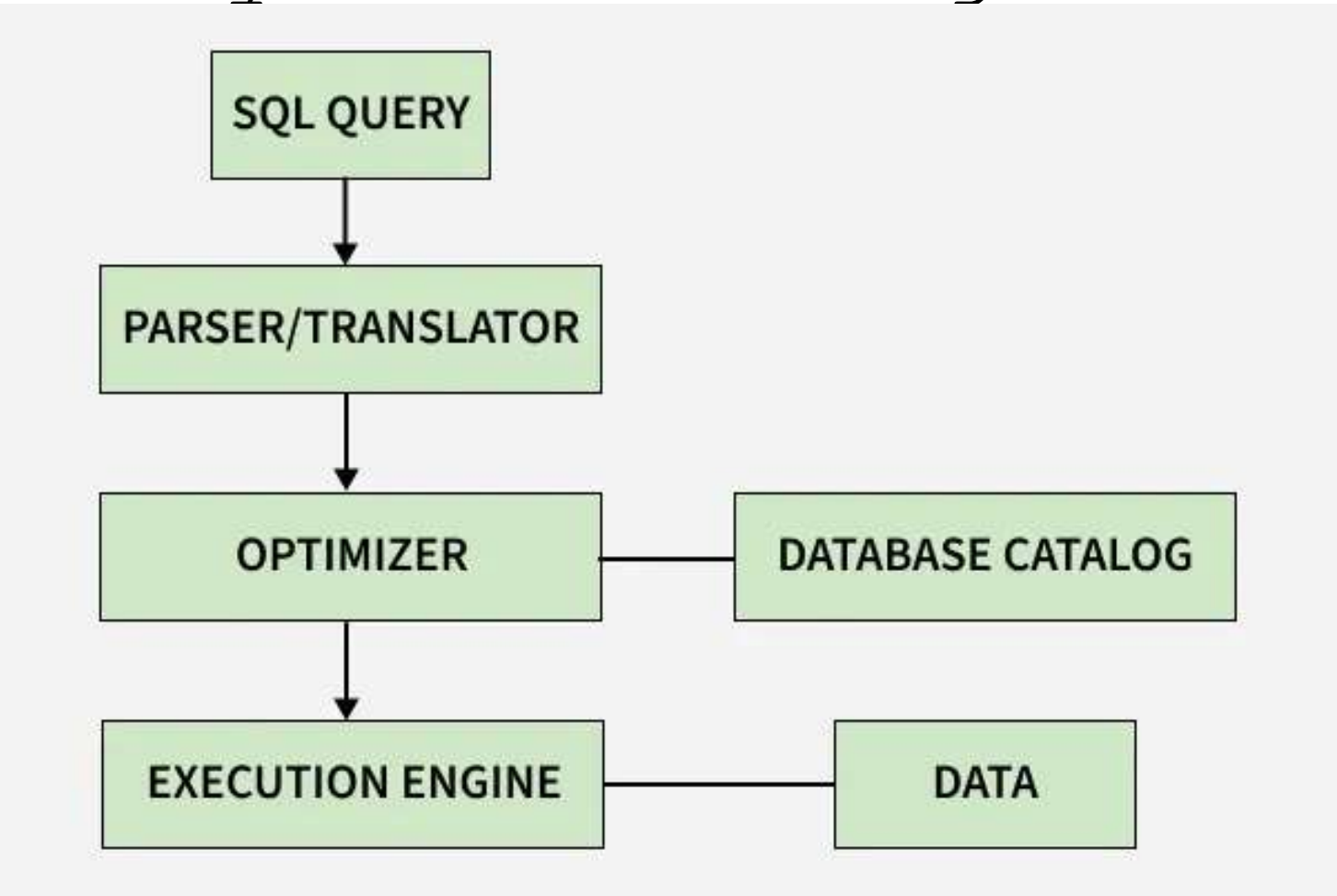
Data Storage and Indexing

Behind the Scenes of SQL Query Execution

SQL Query Processing



Steps

The Query Arrives — Connection and Authentication

- As soon as the query reaches the database server (for example, SELECT name, age FROM users WHERE city = ‘New York’;), it hits the **connection manager** — essentially the database’s front door
  - Authentication

Parsing the SQL — Understanding the Request

- SQL query goes to the **query parser -** breaks the query text into parts; checks the SQL **syntax;** verifies that all referenced tables and columns actually exist; and there is access to them

Optimization — Planning the Best Route

- Most efficient way to execute it - optimizer considers different strategies to get data and decides on the fastest one using statistics about the data
- **Execution plan** — a blueprint of the steps the database will take to get your results

Execution Engine — Carrying Out the Plan

Execution Engine

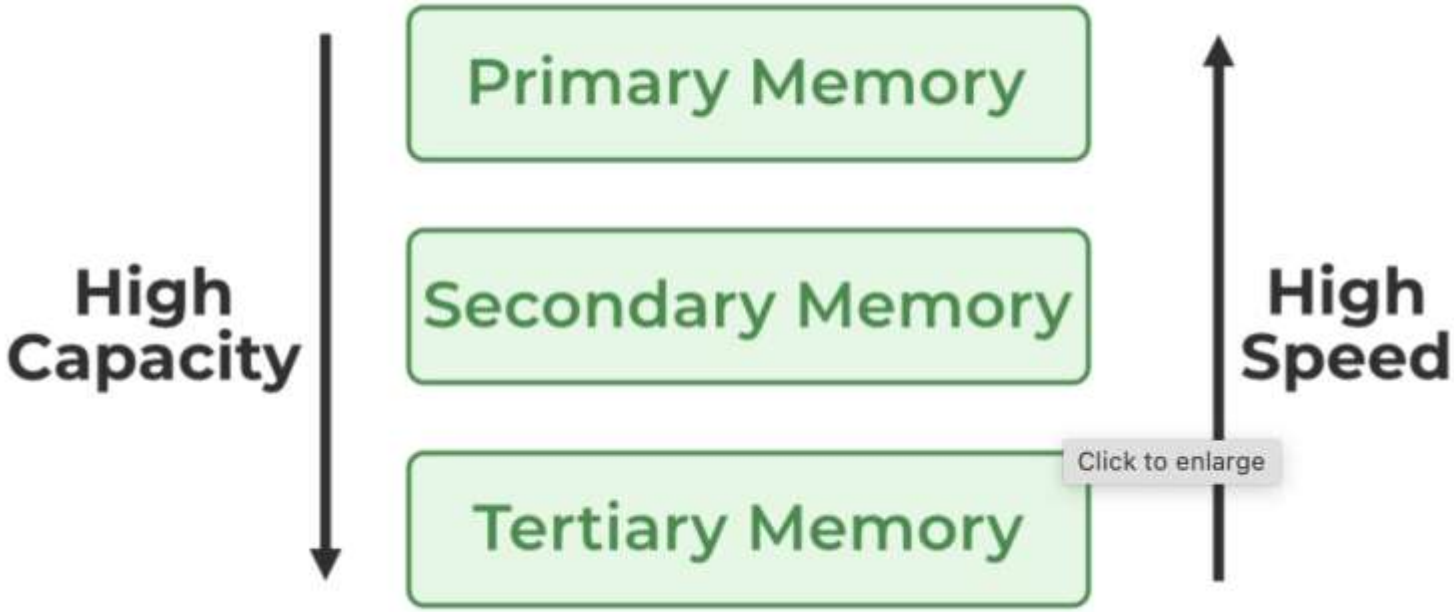
- If the plan says “fetch all users from New York via an index,” - the **execution engine will ask the storage engine** to do that, then take those results and maybe filter out just the name and age fields

Storage Engine — Retrieving Data and Ensuring Integrity

- The **storage engine** is responsible for the actual data retrieval
- The storage engine interacts with the disk and memory, and also makes sure that the database’s integrity is maintained while the query runs
- Ensures that query runs safely by using **transactions** (all-or-nothing execution) and applying **locks** to prevent conflicts with other queries
- The storage engine also checks its in-memory **cache** for the required data (for speed) and goes to disk if needed, logging any changes along the way for recovery

The records in databases are stored in **file formats**

Physically, the data is stored in electromagnetic format on a device



Primary Memory

- This is directly accessible by the central processing unit

- According to performance and speed, the primary memory devices are ***the fastest devices,*** and this feature is in direct correlation with their capacity
- The size of these devices is considerably smaller and they are volatile.
- Usually more expensive due to their increased speed and performance.
- **Cache Memory:** Cache Memory is a special very high-speed memory. It is used to speed up and synchronize with a high-speed CPU

Secondary Memory

- Devices that can be accessed for storing data that will be needed at a later point in time for various purposes or database actions
- The size of this group of devices is noticeably larger than the primary devices and smaller than the tertiary devices.

- Flash memory, also known as flash storage, is a type of nonvolatile memory that erases data in units called blocks and rewrites data at the byte level

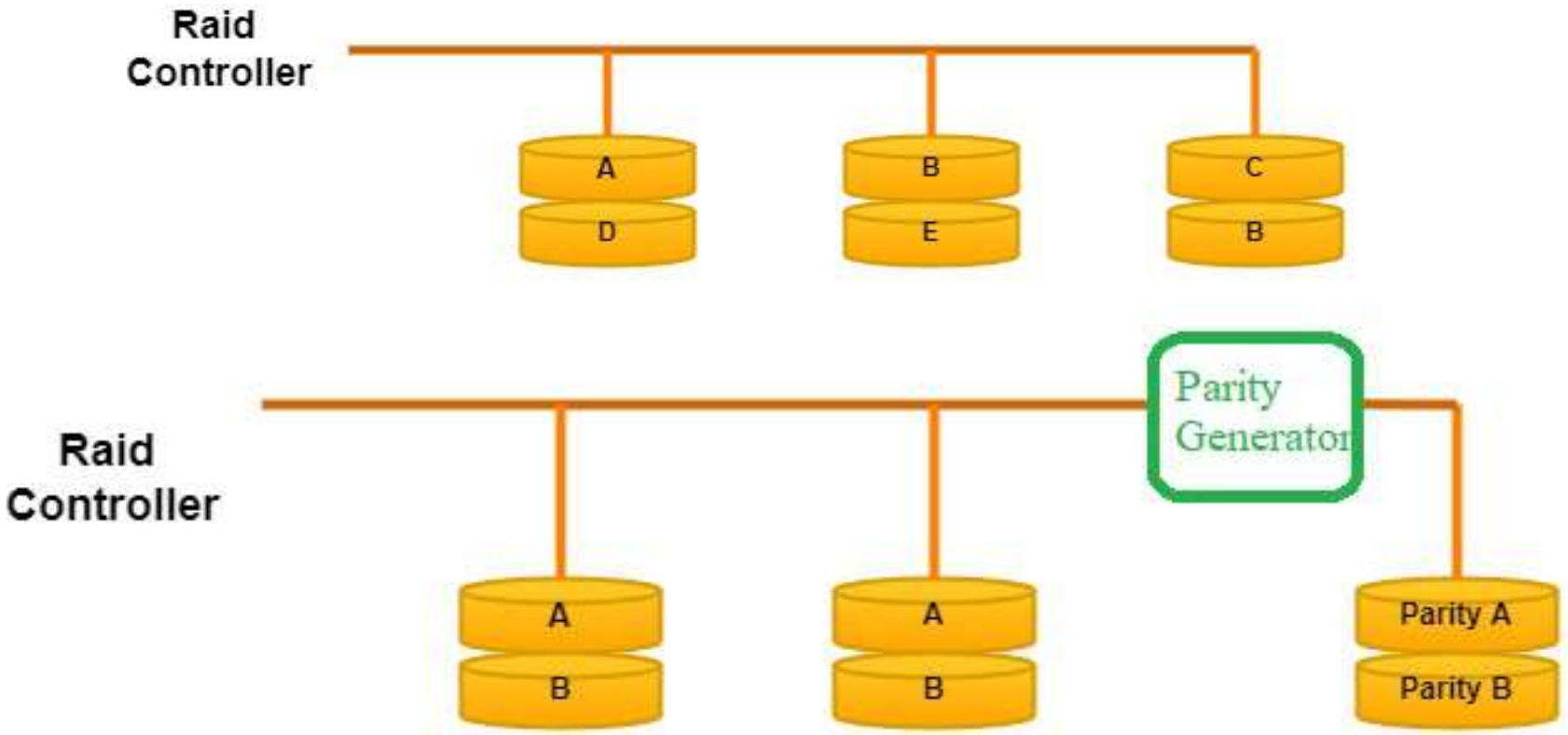
Flash memory is widely used for storage and data transfer in consumer devices, enterprise systems, and industrial applications

Unlike traditional hard drives, flash memories are able to retain data even after the power has been turned off

Magnetic Disk Storage: A Magnetic Disk is a type of secondary memory that is a flat disc covered with a magnetic coating to hold information

Redundant Array of Independent Disks

In the Redundant Array of Independent Disks technology, two or more secondary storage devices are connected so that the devices operate as one st



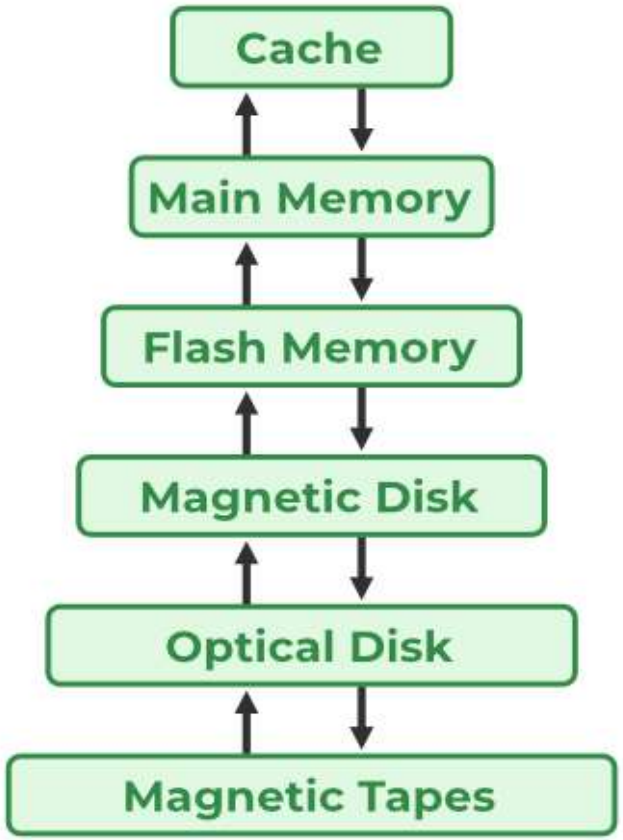
Tertiary Memory

- Tertiary Memory refers to devices that can hold a large amount of data without being constantly connected to the server or the peripherals
- The cost of tertiary storage is lower than primary and secondary
- Means to make a backup of data, this type of storage is commonly used for making copies from servers and databases

- **Optical Storage:** It is a type of storage where reading and writing are to be performed with the help of a laser

- **Tape Storage:** Tape Storage is a type of storage data where we use magnetic tape to store data

Storage Device Hierarchy



Accessing Data

- **Access Types:** This refers to the **type of access**, such as value-based search, range access, etc.
- **Access Time:** It refers to the **time** needed to find a particular data element or set of elements.
- **Insertion Time:** It refers to the time taken to **find the appropriate space and insert new data.**
- **Deletion Time:** Time taken to **find an item an**d delete it as well as update the index structure.
- **Space Overhead:** It refers to the additional **space** required by the index.

File Organization

- The database is stored as a collection of *files*. Each file is a sequence of *records.* A record is a sequence of fields.
- One approach
  - Assume record size is fixed
  - Each file has records of one particular type only
  - Different files are used for different relations

  This case is easiest to implement
- We assume that records are smaller than a disk block

Fixed-Length Records

- Simple approach:
  - Store record *i* starting from byte $n * (i - 1)$, where *n* is the size of each record.
  - Record access is simple but records may cross blocks
    - **Modification: do not allow records to cross block boundaries**

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Fixed-Length Records

- Deletion of record *i:* alternatives*:*
  - **move records *i* + 1, . . ., *n* to *i, . . . , n* – 1**
  - move record *n* to *i*
  - **do not move records, but link all free records on a *free list***

Record 3 deleted

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Fixed-Length Records

- Deletion of record *i:* alternatives*:*
  - move records $i + 1, \ldots, n$ to $i, \ldots, n - 1$
  - **move record *n* to *i***
  - do not move records, but link all free records on a *free list*

Record 3 deleted and replaced by record 11

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 11	98345	Kim	Elec. Eng.	80000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000

Fixed-Length Records

- Deletion of record *i:* alternatives*:*
  - move records $i + 1, \ldots, n$ to $i, \ldots, n - 1$
  - move record n to i
  - **do not move records, but link all free records on a *free list***

header				
record 0	10101	Srinivasan	Comp. Sci.	65000
record 1				
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4				
record 5	33456	Gold	Physics	87000
record 6				
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Organization of Records in Files

- **Heap** – record can be placed anywhere in the file where there is space
- **Sequential** – store records in sequential order, based on the value of the search key of each record
- In a **multitable clustering file organization** records of several different relations can be stored in the same file
  - Motivation: store related records on the same block to minimize I/O
- **B^+-tree file organization**
  - Ordered storage even with inserts/deletes
  - More on this in Chapter 14
- **Hashing** – a hash function computed on search key; the result specifies in which block of the file the record should be placed
  - More on this in Chapter 14

Heap File Organization

- Records can be placed anywhere in the file where there is free space
- Records usually do not move once allocated
- Important to be able to efficiently find free space within file
- **Free-space map**
  - Array with 1 entry per block. Each entry is a few bits to a byte, and records fraction of block that is free
  - In example below, 3 bits per block, value divided by 8 indicates fraction of block that is free

4	2	1	4	7	3	6	5	1	2	0	1	1	0	5	6

Sequential File Organization

- Suitable for applications that require sequential processing of the entire file
- The records in the file are ordered by a search-key

10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	

Sequential File Organization (Cont.)

- Deletion – use pointer chains
- Insertion –locate the position where the record is to be inserted
  - if there is free space insert there
  - if no free space, insert the record in an overflow block
  - In either case, pointer chain must be updated
- Need to reorganize the file from time to time to restore sequential order

10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	

32222	Verdi	Music	48000	

Multitable Clustering File Organization

Store several relations in one file using a **multitable clustering** file organization

department

dept_name	*building*	*budget*
Comp. Sci.	Taylor	100000
Physics	Watson	70000

instructor

ID	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000

multitable clustering of *department* and *instructor*

Comp. Sci.	Taylor	100000	
10101	Srinivasan	Comp. Sci.	65000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000
Physics	Watson	70000	
33456	Gold	Physics	87000

Multitable Clustering File Organization (cont.)

- Good for queries involving *department* $\bowtie$ *instructor*, and for queries involving one single department and its instructors
- bad for queries involving only *department*
- results in variable size records
- Can add pointer chains to link records of a particular relation

Partitioning

- **Table partitioning**: Records in a relation can be partitioned into smaller relations that are stored separately
- E.g., *transaction* relation may be partitioned into *transaction_2018, transaction_2019, etc.*

- Queries written on *transaction* must access records in all partitions
  - Unless query has a selection such as *year*=2019, in which case only one partition in needed
- Partitioning
  - Reduces costs of some operations such as free space management
  - Allows different partitions to be stored on different storage devices
    - E.g., *transaction* partition for current year on SSD, for older years on magnetic disk

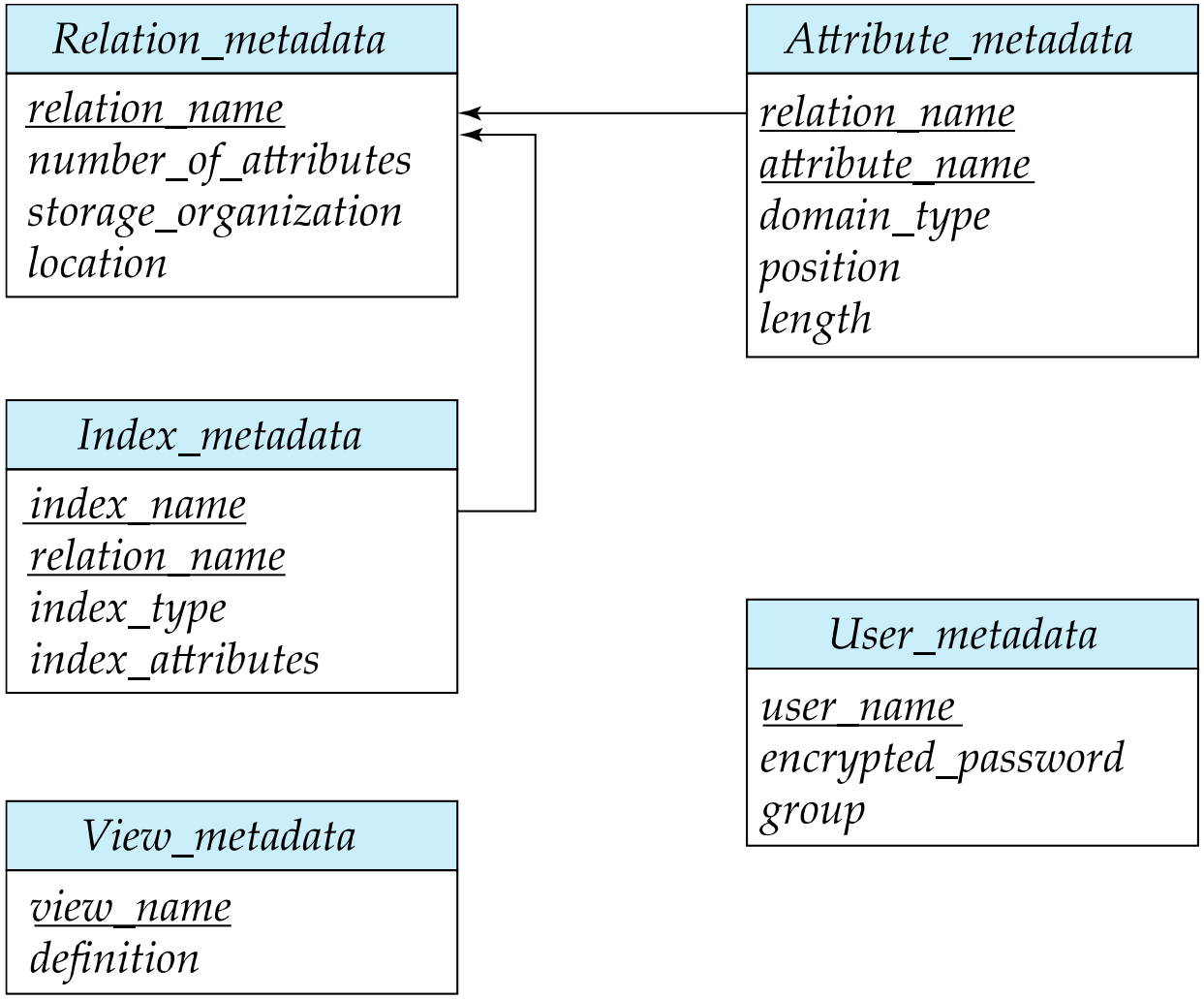
Data Dictionary Storage

The **Data dictionary** (also called **system catalog**) stores **metadata**; that is, data about data, such as

- Information about relations
  - names of relations
  - names, types and lengths of attributes of each relation
  - names and definitions of views
  - integrity constraints
- User and accounting information, including passwords
- Statistical and descriptive data
  - number of tuples in each relation
- Physical file organization information
  - How relation is stored (sequential/hash/…)
  - Physical location of relation

Relational Representation of System Metadata

- Relational representation on disk
- Specialized data structures designed for efficient access, in memory



Indexing

- Many / Most queries reference only a small portion of the records in a file
  - Example –
    - Find all instructors in Physics Department
    - Find courses registered by student ID – 21200
- Inefficient for the system to read every record to check the department value
- **An Index in a DBMS works the same way as an index of words work for books –** - To look up a word - find it in the index – go to the page number
    - Words are in sorted order
- DBMS would look up an index to find the block in which the record is stored

Basic Concepts

- Indexing mechanisms used to speed up access to desired data.
  - E.g., author catalog in library
- **Search Key** - attribute to set of attributes used to look up records in a file.
- An **index file** consists of records (called **index entries**) of the form

search-key	pointer

- Index files are typically much smaller than the original file
- Two basic kinds of indices:
  - **Ordered indices:** search keys are stored in sorted order
  - **Hash indices:** search keys are distributed uniformly across “buckets” using a “hash function”.

Ordered Indices

- In an **ordered index,** index entries are stored sorted on the search key value.

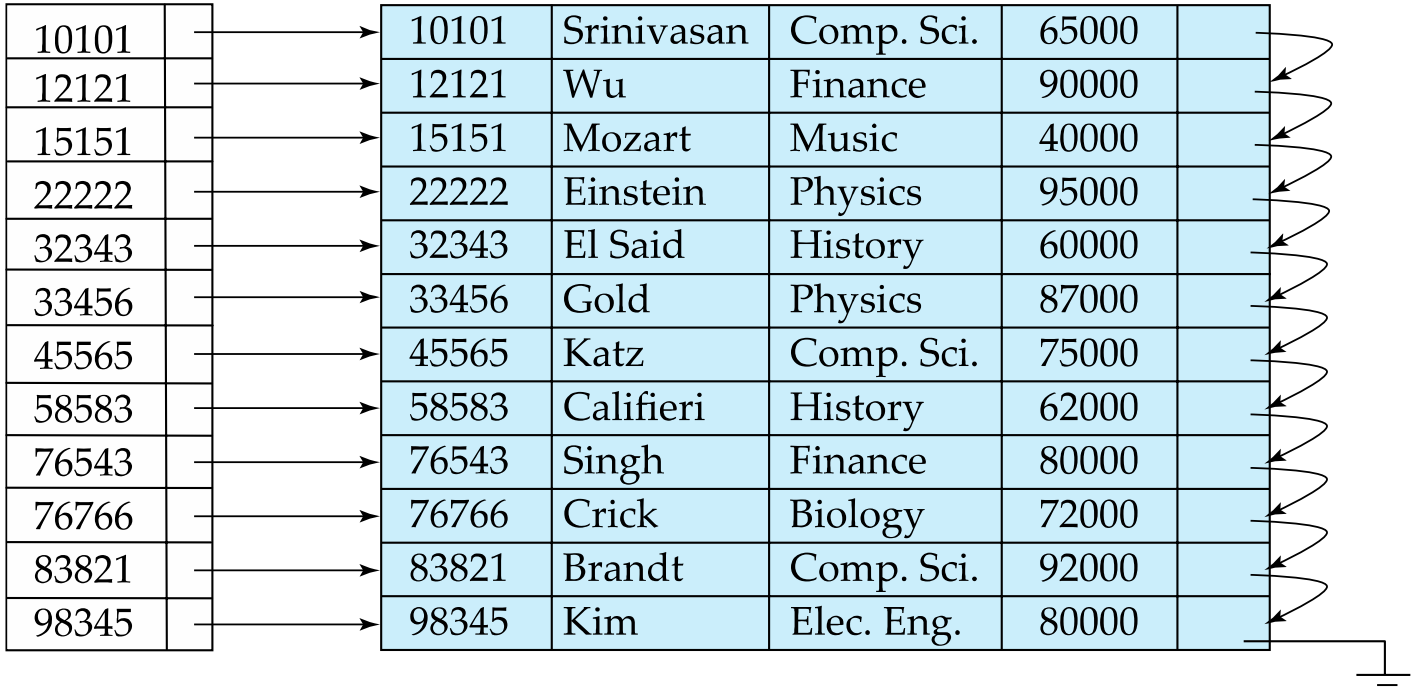
- **Clustering index:** in a sequentially ordered file, the index whose search key specifies the sequential order of the file.
  - Also called **primary index**
  - The search key of a primary index is usually but not necessarily the primary key.

- **Secondary index**: an index whose search key specifies an order different from the sequential order of the file. Also called **nonclustering index.**

- **Index-sequential file:** sequential file ordered on a search key, with a clustering index on the search key.

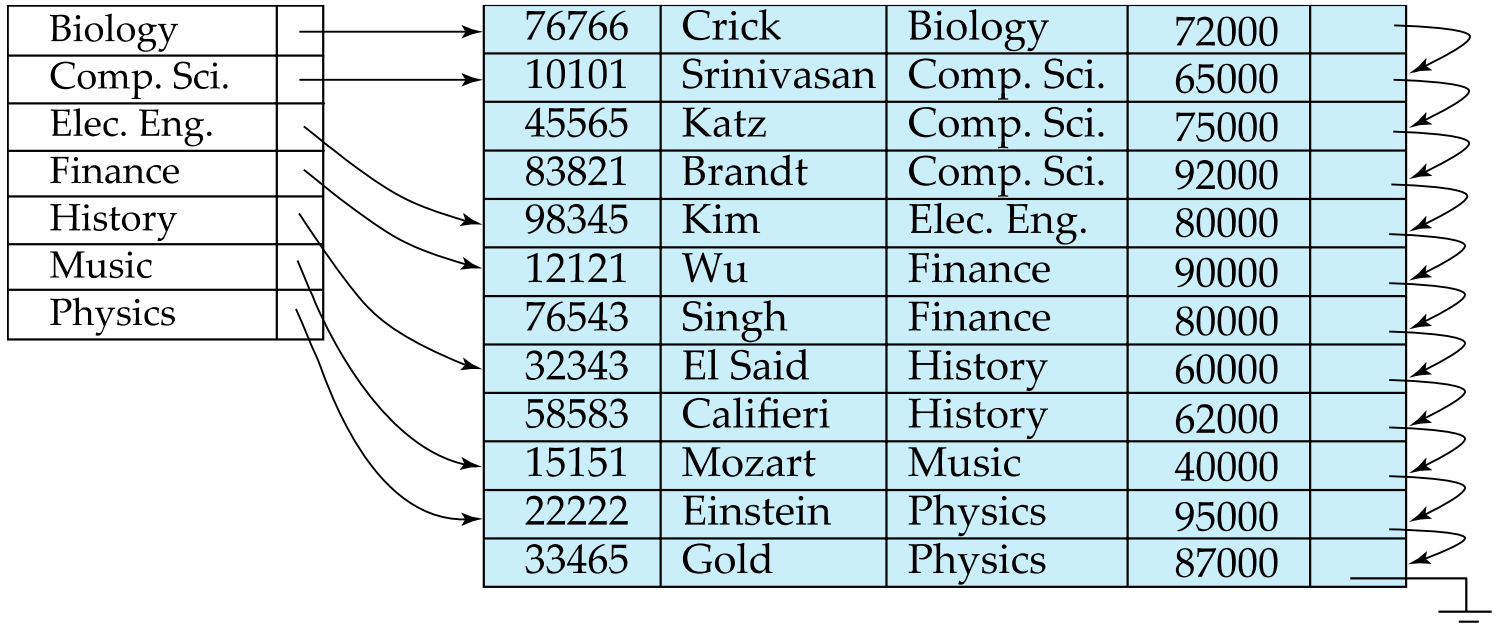
Dense Index Files

- **Dense index** — Index record appears for every search-key value in the file.
- E.g. index on *ID* attribute of *instructor* relation



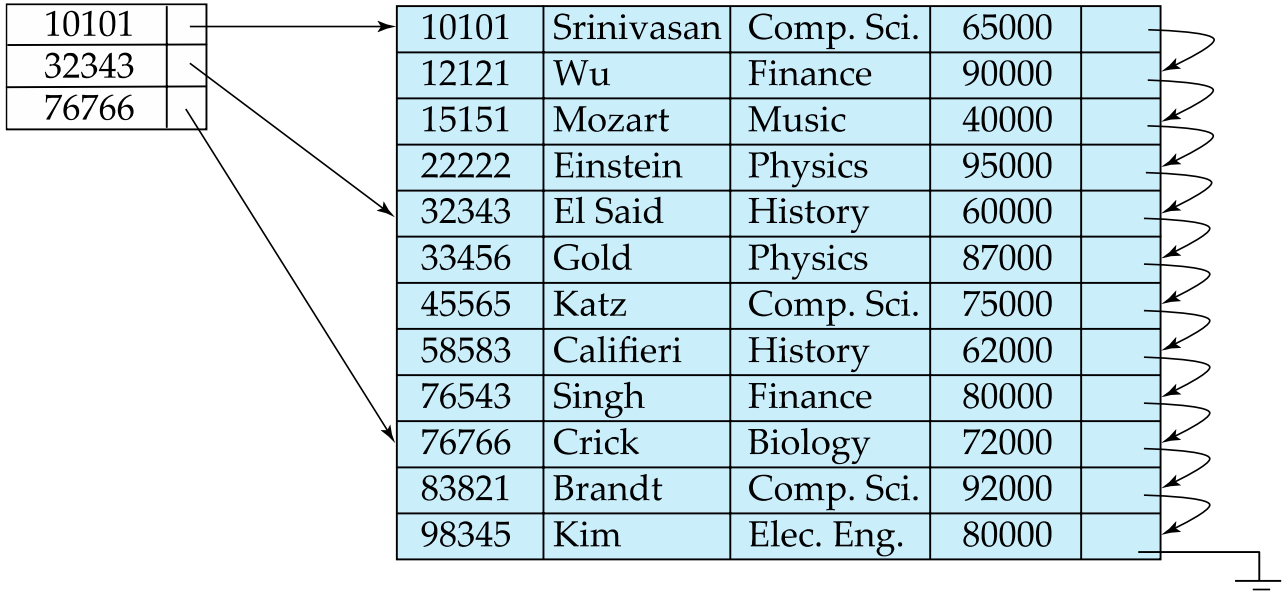
Dense Index Files (Cont.)

- Dense index on *dept_name*, with *instructor* file sorted on *dept_name*



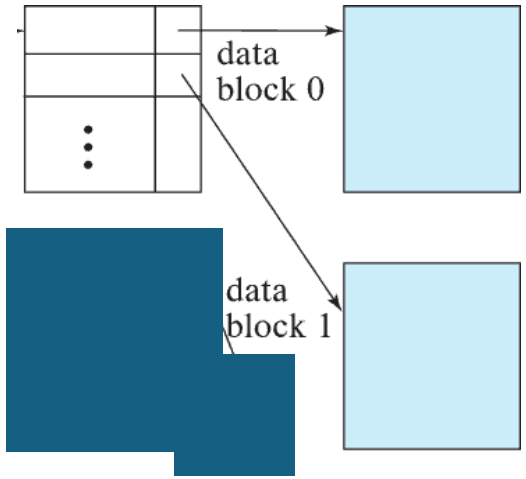
Sparse Index Files

- **Sparse Index**: contains index records for only some search-key values.
  - Applicable when records are sequentially ordered on search-key
- To locate a record with search-key value *K* we:
  - Find index record with largest search-key value < *K*
  - Search file sequentially starting at the record to which the index record points



Sparse Index Files (Cont.)

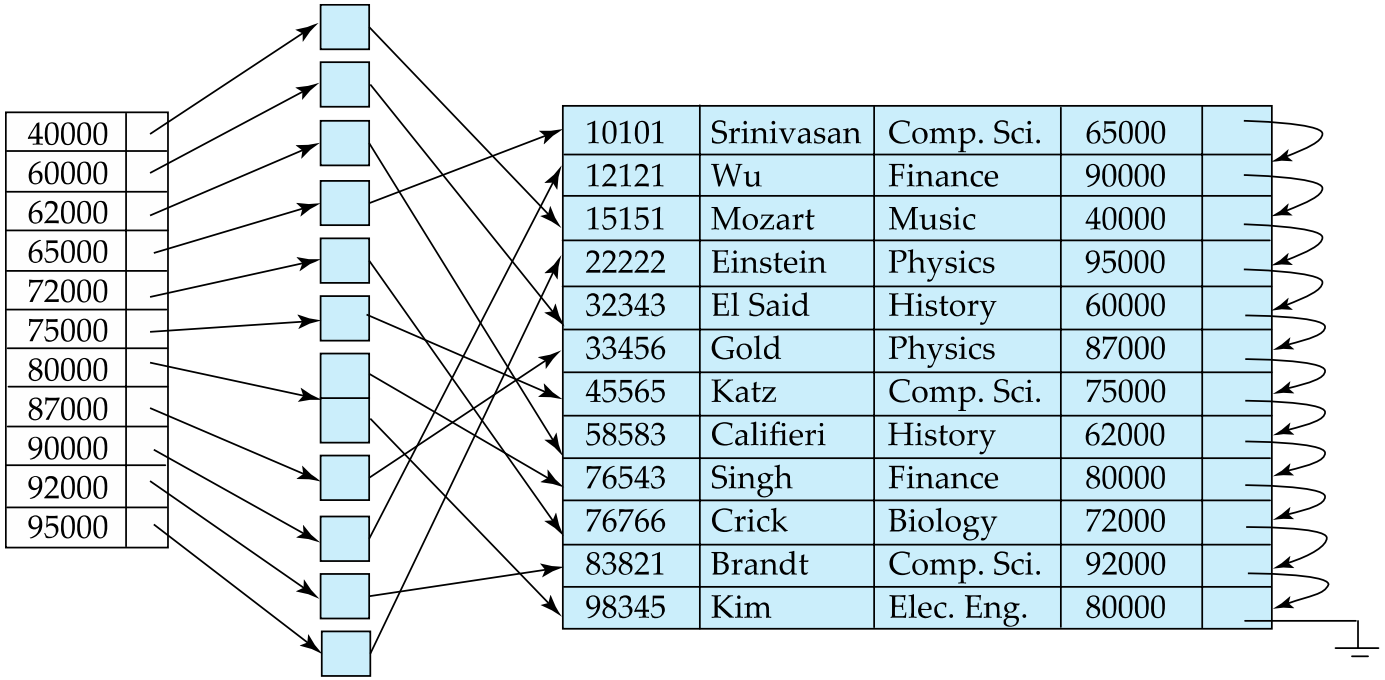
- Compared to dense indices:
  - Less space and less maintenance overhead for insertions and deletions.
  - Generally slower than dense index for locating records.
- **Good tradeoff**:
  - for clustered index: sparse index with an index entry for every block in file, corresponding to least search-key value in the block.



  - For unclustered index: sparse index on top of dense index (multilevel index)

Secondary Indices Example

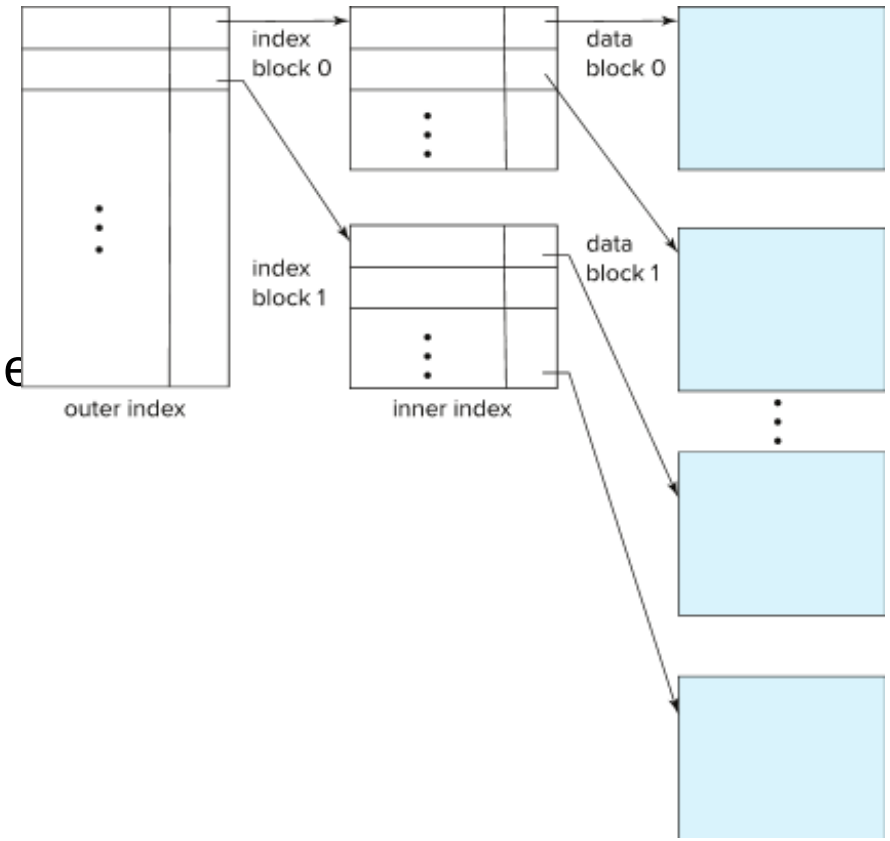
- Secondary index on salary field of instructor



- Index record points to a bucket that contains pointers to all the actual records with that particular search-key value.
- Secondary indices have to be dense

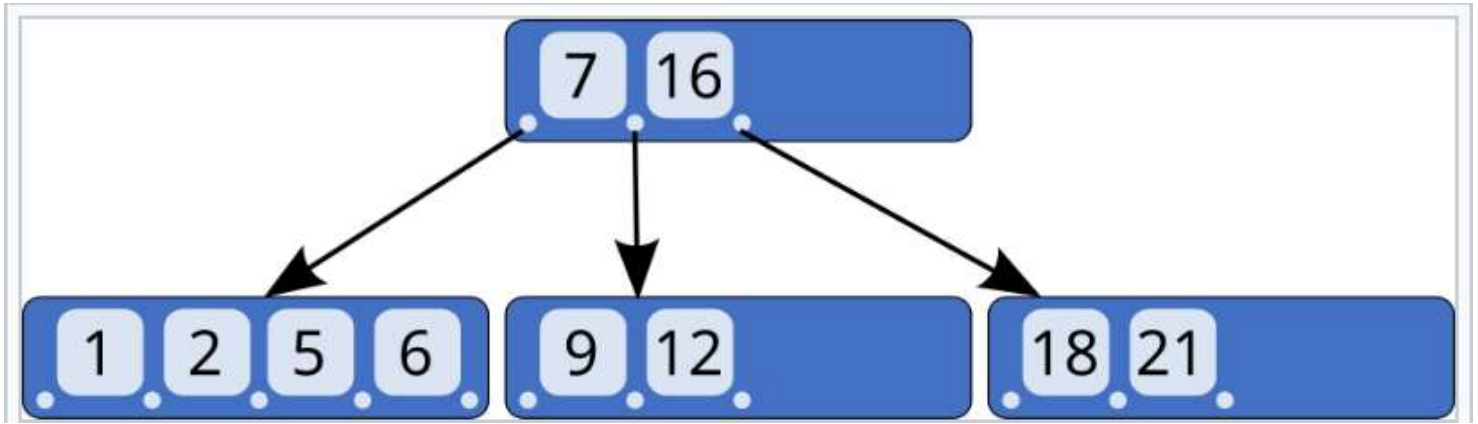
Multilevel Index

- If index does not fit in memory, access becomes expensive.
- Solution: treat index kept on disk as a sequential file and construct a sparse index on it.
  - outer index – a sparse index of the basic index
  - inner index – the basic index file
- If even outer index is too large to fit in main memory, yet anothe level of index can be created, and so on.
- Indices at all levels must be updated on insertion or deletion from the file.



B-Tree

- **B-tree** is a self-balancing tree that maintains sorted data and allows searches, sequential access, insertions, and deletions in logarithmic time



B^+-Tree Index Files

- A **B+ Tree** is a self-balancing **multi-level index** tree structure used in **database indexing** and **file systems**.

- **Purpose**: Helps in **fast searching, insertion, deletion**, and efficient **range queries**.

- In the B+ tree, the internal nodes do not store any pointers to records
- Overall fewer pointers - each node can hold more keys - the trees are shallower and thus faster to search.
- All pointers to records are stored in the leaf nodes
  - In addition, a leaf node may include a pointer to the next leaf node to speed up sequential access

Structure of a B+ Tree

Properties:

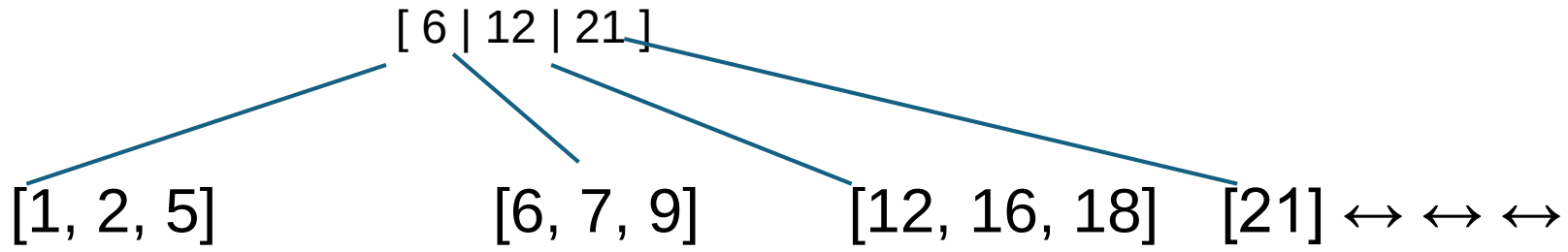
1. **Order (d)**: Maximum number of children per node = **d**.
2. **Balanced Structure**: All **leaf nodes** are at the **same level**.
3. **Internal Nodes**: Store **keys** but no actual data.
4. **Leaf Nodes**:
    1. Store **actual records (or pointers)**.
    2. Connected as a **linked list** (useful for range queries).
5. **Root Node**: Can have **2 to d** children (except for the first root).

B^+-Tree Index Files (Cont.)

A B^+-tree is a rooted tree satisfying the following properties:

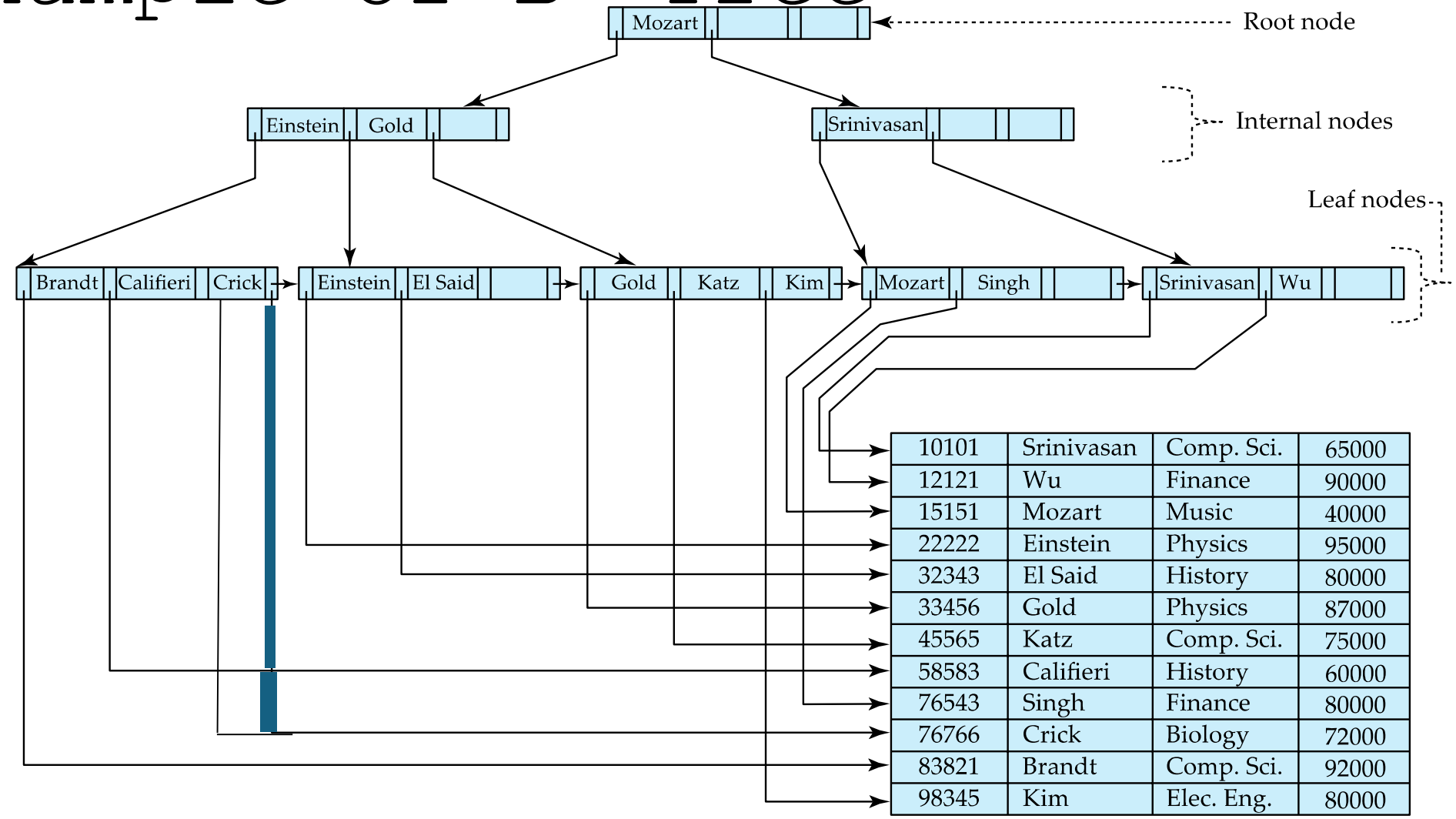
- All paths from root to leaf are of the same length
- Each node that is not a root or a leaf has between $\lceil n/2 \rceil$ and n children – ***Order d can be defined***
- A leaf node has between $\lceil (n-1)/2 \rceil$ and $n-1$ values
- Special cases:
  - If the root is not a leaf, it has at least 2 children.
  - If the root is a leaf (that is, there are no other nodes in the tree), it can have between 0 and $(n-1)$ values.

- Let’s use a B+ tree where:
- Each internal node can have up to 3 keys (i.e., up to 4 children)
- Each leaf node can have up to 3 keys



-

Example of B^+-Tree



10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	80000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	60000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

B^+-Tree Node Structure

- Typical node

P_1	K_1	P_2	…	$P_{n\text{-}1}$	$K_{n\text{-}1}$	P_n

  - K_i are the search-key values
  - P_i are pointers to children (for non-leaf nodes) or pointers to records or buckets of records (for leaf nodes).
- The search-keys in a node are ordered

$$K_1 < K_2 < K_3 < \ldots < K_{n-1}$$

(Initially assume no duplicate keys, address duplicates later)

Non-Leaf Nodes in B^+-Trees

- Non leaf nodes form a multi-level sparse index on the leaf nodes.
- For a non-leaf node with *m* pointers:
  - All the search-keys in the subtree to which P_1 points are less than K_1
  - For $2 \leq i \leq n - 1$, all the search-keys in the subtree to which P_i points have values greater than or equal to K_{i-1} and less than K_i
  - All the search-keys in the subtree to which P_n points have values greater than or equal to K_{n-1}
  - General structure

P_1	K_1	P_2	…	P_{n-1}	K_{n-1}	P_n

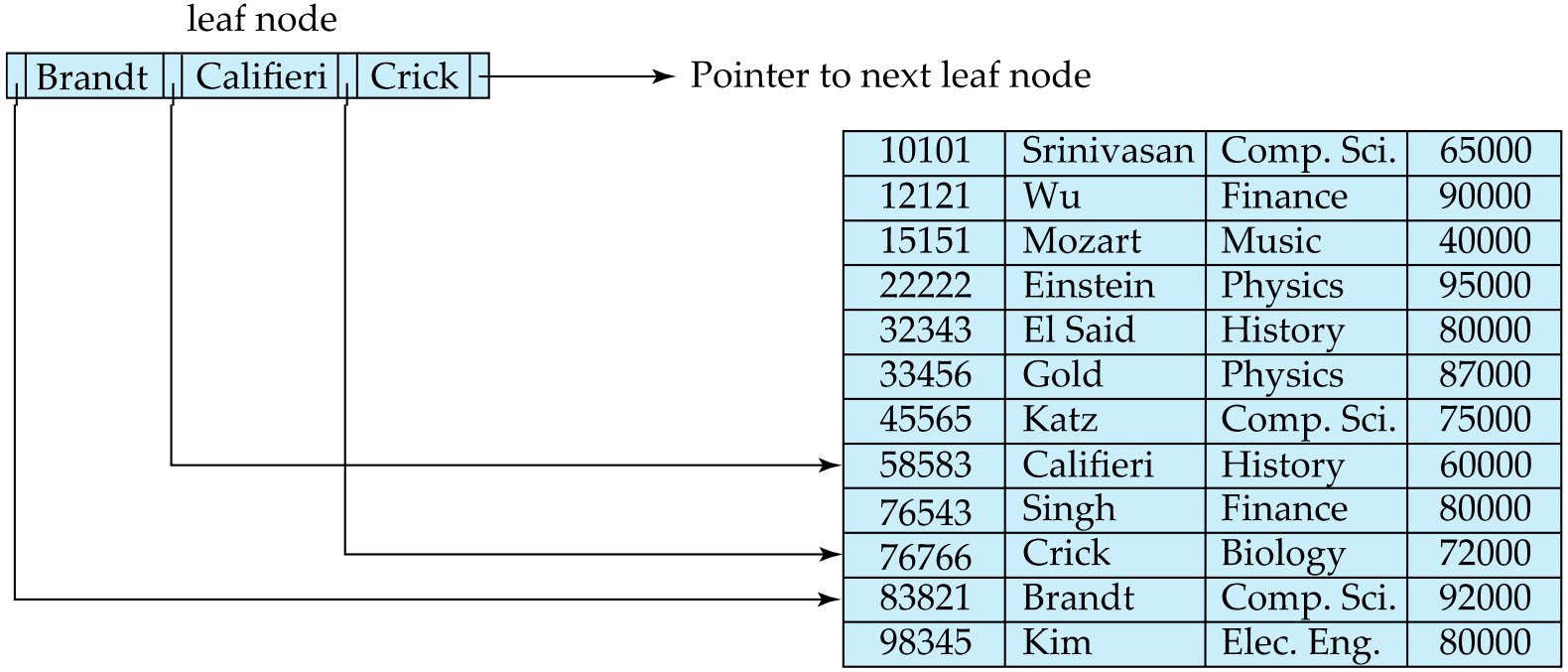
Index Evaluation Metrics

- Access types supported efficiently. E.g.,
  - Records with a specified value in the attribute
  - Records with an attribute value falling in a specified range of values.

- Access time
- Insertion time
- Deletion time
- Space overhead

Leaf Nodes in B$^+$-Trees

- For i = 1, 2, . . ., n–1, pointer P_i points to a file record with search-key value K_i,
- If L_i, L_j are leaf nodes and $i < j$, L_i's search-key values are less than or equal to L_j's search-key values
- P_n points to next leaf node in search-key order



10101	Srinivasan	Comp. Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
32343	El Said	History	80000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
58583	Califieri	History	60000
76543	Singh	Finance	80000
76766	Crick	Biology	72000
83821	Brandt	Comp. Sci.	92000
98345	Kim	Elec. Eng.	80000

Operations on B+ Trees

(a) Searching

- Start from the **root**.
- Compare key with **internal node values**.
- Follow the **appropriate child pointer**.
- Once at the **leaf node**, perform a **linear search**.

- **Time Complexity**: **$O(\log_d N)$**

(b) Insertion

- Find the **correct leaf node**.
- Insert the key **in sorted order**.
- If the leaf node **overflows**, **split** it.
- Propagate the split **upward** if needed.

Operations

(c) Deletion

- Find the **leaf node** containing the key.
- Remove the key.
- If the node **underflows**, **borrow** from sibling or **merge** nodes.
- Adjust **internal nodes** if needed.

Example

- Construct a **B+ Tree of order 4** and insert the keys: 10, 20, 5, 6, 12, 30, 7, 17
- Maximum number of children per node = **4**

```
[10]
```

```
[5, 6, 10, 20]
```

- Insert 12 - Node Split! - **Left Leaf:** [5, 6] - **Right Leaf:** [10, 12, 20] - **middle key (10)** moves **up** to the root node.

```
      [10]
     /    \
[5, 6]  [10, 12, 20]
```

10, 20, 5, 6, 12, 30, 7, 17

```
        [10]
       /    \
  [5, 6]  [10, 12, 20, 30]
```

```
        [10]
       /    \
  [5, 6, 7]  [10, 12, 20, 30]
```

```
      [10, 17]
     /    |    \
[5, 6, 7] [10, 12] [17, 20, 30]
```

Insert 17 – needs second Node Split!

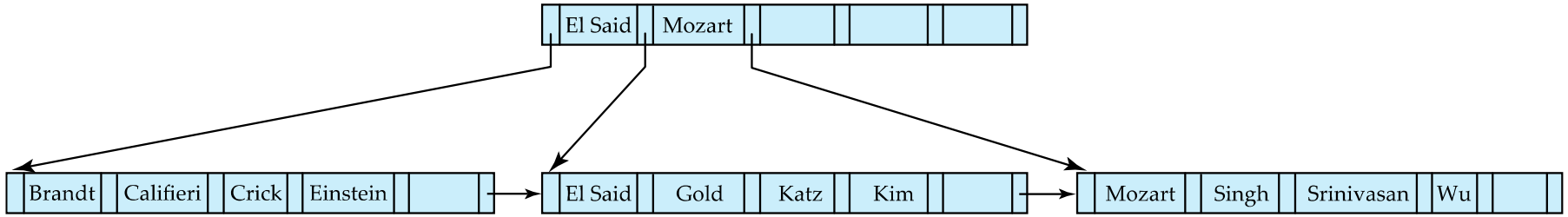
•The right leaf node [10, 12, 20, 30] is full, so it splits:

- Left Leaf: [10, 12]
- Right Leaf: [17, 20, 30]

•The middle key (17) moves up to the root.

Example of B^+-tree

- B^+-tree for *instructor* file (n = 6)



- Leaf nodes must have between 3 and 5 values

($\lceil (n-1)/2 \rceil$ and $n-1$, with n = 6).

- Non-leaf nodes other than root must have between 3 and 6 children ($\lceil (n/2 \rceil$ and n with n =6).

- Root must have at least 2 children.

Observations about B^+-trees

- Since the inter-node connections are done by pointers, “logically” close blocks need not be “physically” close.
- The non-leaf levels of the B^+-tree form a hierarchy of sparse indices.
- The B^+-tree contains a relatively small number of levels
  - Level below root has at least $2 * \lceil n/2 \rceil$ values
  - Next level has at least $2 * \lceil n/2 \rceil * \lceil n/2 \rceil$ values
- If there are K search-key values in the file, the tree height is no more than $\lceil \log_{\lceil n/2 \rceil}(K) \rceil$
  - ***thus searches can be conducted efficiently.***
- Insertions and deletions to the main file can be handled efficiently, as the index can be restructured in logarithmic time